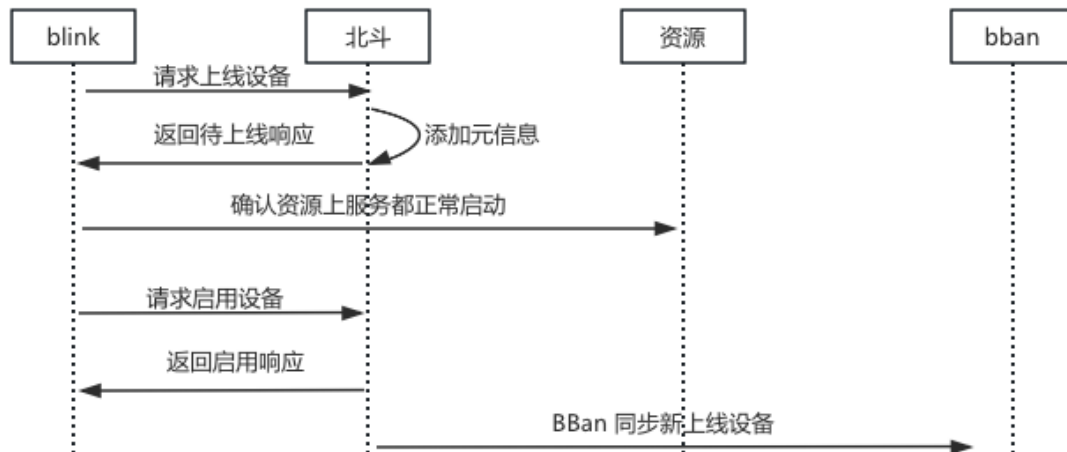


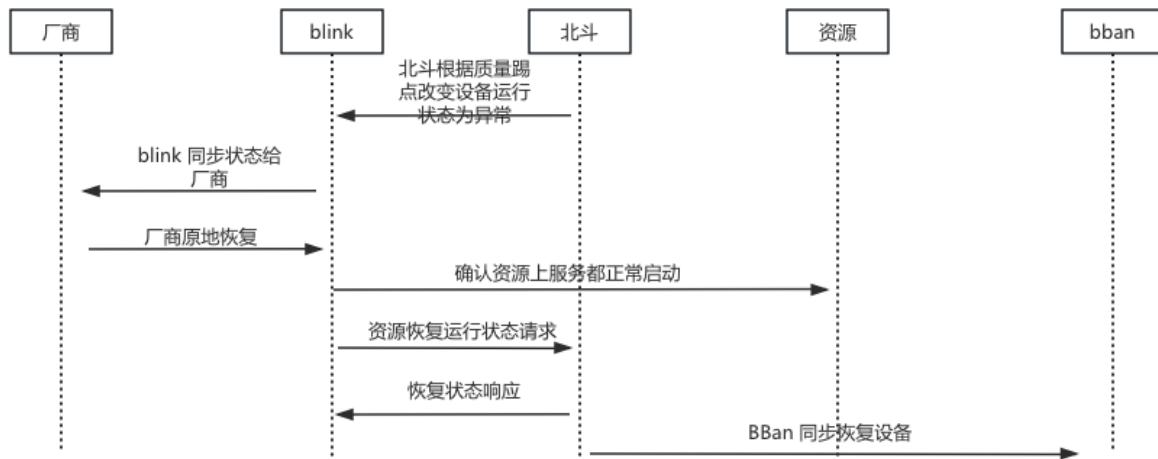
北斗资源上下线&踢点逻辑

与当前 CDN相关的廉价资源上下线流程和踢点流程如下

新资源交付流程

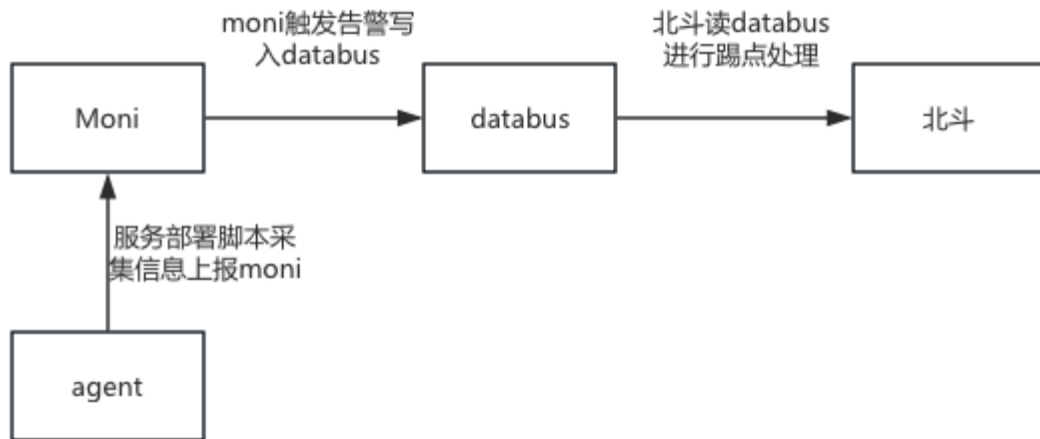


厂商直接恢复流程



如资源故障厂商选择下线维修，则再次交付时会走新资源交付流程

踢点架构逻辑



流程

生产侧：资源部署时会有脚本监控，监测系统数据并上报moni，moni写入databus

消费侧：北斗消费databus消息，对于触发下面告警条件的节点进行下线处罚

触发条件

丢包率、重传率、磁盘故障率、CPU负载、根目录、IP冲突

rule_id	告警类型	业务方	执行动作	描述	操作
857263	系统	北斗	直接下线	mCDN-hostname异常-自动下线	编辑 删除
859556	系统	北斗	告警	mCDN-内存低于64-自动下线	编辑 删除
859660	系统	北斗	直接下线	mCDN-CPU频率低于1200-自动下线	编辑 删除
859681	系统	北斗	告警	mCDN-cpu核心低于32-自动下线	编辑 删除
859687	系统	移动端	直接下线	MOB-CPU负载大于80-自动下线	编辑 删除
859748	系统	移动端	直接下线	MOB-磁盘故障率大于50%-自动下线	编辑 删除
859809	网络	移动端	直接下线	MOB-重传率大于12%-自动下线	编辑 删除
859813	网络	移动端	直接下线	MOB-丢包率大于12%-自动下线	编辑 删除
3084592	质量	移动端	直接下线	MOB-大节点错误率异常-自动下线	编辑 删除
860072	网络	视频云	直接下线	BVC-丢包率大于15%-自动下线	编辑 删除
860088	网络	视频云	直接下线	BVC-重传率大于15%-自动下线	编辑 删除
860106	系统	视频云	直接下线	BVC-磁盘故障率大于50%-自动下线	编辑 删除
860109	系统	视频云	直接下线	BVC-CPU负载大于80-自动下线	编辑 删除
860110	系统	视频云	直接下线	BVC-根目录大于80%-自动下线	编辑 删除
860111	系统	视频云	直接下线	BVC-系统盘util大于75-自动下线	编辑 删除
860117	网络	视频云	直接下线	BVC-IP冲突-自动下线	编辑 删除
860157	网络	视频云	直接下线	BVC-线路可用率小于50%-自动下线	编辑 删除
860151	网络	移动端	直接下线	MOB-线路可用率小于50%-自动下线	编辑 删除
860289	网络	移动端	直接下线	MOB-IP冲突-自动下线	编辑 删除
1910012	北斗	北斗	直接下线	MOB-mnet业务异常-直接下线	编辑 删除

监控地址

<https://moni.bilibili.co/d/hMWqO8nGz/mcdn-e88a82-e782b9-e79b91-e68ea7?orgId=1&...>

其中与网络相关的主要有丢包率和重传率

对于不同的业务其阈值不同

BVC：

https://cloud.bilibili.co/alarm/config/alertRule/detail?id=860072&activeTab=1&start_tim...

描述	PromSql	操作符	阈值
BVC-丢包率大于15%-自动下线	avg by(app, mcdn_identity) (mcdn_network_quality{app=~"mcdn",mcdn_identity=~"mcdn-.*(v live bvcmix)-.*"})	>=	15

关闭

MOB :

https://cloud.bilibili.co/alarm/config/alertRule/detail?id=859813&activeTab=1&start_tim...

触发条件

描述	PromSql	操作符	阈值
MOB-丢包率大于12%-自动下线	avg by(app, mcdn_identity) (mcdn_network_quality{app=~"mcdn",mcdn_identity=~"mcdn-.*(mp2sp)-.*"})	>=	12

关闭

触发告警规则如何写入到databus :

告警平台webhook -> 北斗manba-dispatcher (/api/v1/portal/arrange/webhook) -> databus (Mcdn-dispatcher-01-T)

1 , 告警平台监控踢点事件 :

<https://cloud.bilibili.co/alarm/delivery/subscribe/list?name=%E5%8C%97%E6%96%97>

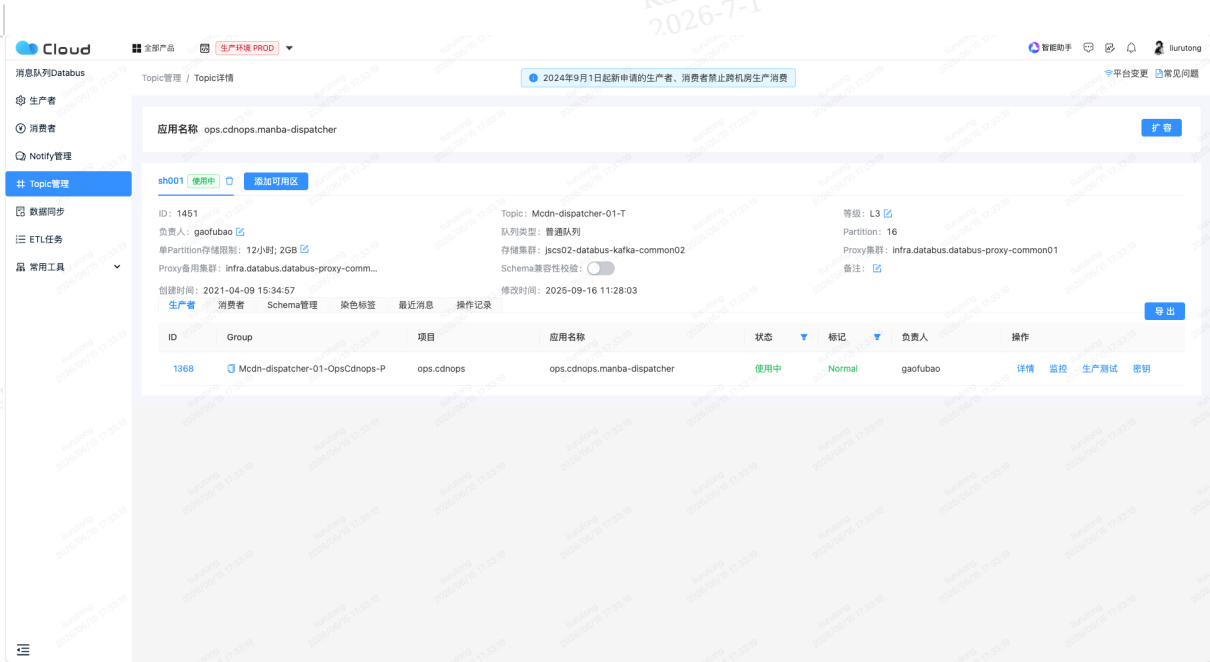
ID	订阅名称	订阅方式	业务	启用状态	创建人	更新时间	操作
600	北斗 mnet 业务异常	webhook	PCDN	启用	dongbohao01	2025-05-13 17:32:47	详情 删除
459	北斗 mcdn 线路订阅	webhook	北斗	启用	v_dongbohao	2024-04-19 14:49:33	详情 删除
455	北斗 mcdn 订阅	webhook	网关层	启用	v_dongbohao	2024-04-17 17:43:53	详情 删除
264	北斗MCDN下线规则	webhook	video-cdn	启用	v_ouyangyibai	0001-01-01 08:05:43	详情 删除

2 , 告警触发 , 通过webhook通知北斗应用manba-dispatch : /api/v1/portal/arrange/webhook

3 , 北斗应用manba-dispatch写入到databus , topic : Mcdn-dispatcher-01-T , group :

Mcdn-dispatcher-01-OpsCdnops-P

<https://cloud.bilibili.co/databus/admintopic/detail?topic=Mcdn-dispatcher-01-T&zoneKe...>



4，北斗应用manba-bi读取databus中的内容，调用北斗应用portal接口，下线节点

北斗恢复节点流程：

1，自动恢复

(1) 适用场景

manba-bi服务启动定时任务，每360s扫描所有的下线节点，根据该节点当天被下线的次数决定恢复节点时间：

今日下线次数	自动恢复条件
1 次	距最后一次下线 $\geq$ 30 分钟 且健康检查通过
2 次	距最后一次下线 $\geq$ 2 小时 且健康检查通过
$\geq$ 3 次	不自动恢复，需人工上线

*当前健康检查为空实现，调用 `moni.GetInstance` 后直接返回 `true`，不做任何实质性健康检查

(2) 恢复流程

(a) 满足恢复条件以后，同步调用`portal.ChangeServerStatus(name, false)`，发起http请求：
`PUT /api/v1/portal/pcdn?name=<hostname>`，将节点的`is_disabled`字段设置为`false`，将`dial_ips`清空，不修改其他字段；执行 SQL：

```
1 UPDATE p_instances SET is_disabled=0, dial_ips='', updated_at=NOW() WHERE name=?
```

(b) 异步向 msg_data_all 表插入一条 action_id=400 的记录，标记本次自动上线事件。

(c) 异步发送企业微信通知和blink通知

(d) portal 的缓存刷新 ticker (ticker.go:19) 每 30 秒执行一次 refreshMCDN 等函数，从 dao.GetLiveMCDNs 重新查库：

```
1  -- dao/mcdn.go:11
2  updated_at > NOW() - INTERVAL 3 MINUTE
3  AND is_deleted = 0
4  AND dial_ips != ""
5  AND type = 'v'
```

此时 is_disabled 刚被置为 false，但 dial_ips 也同时被清空了，节点不会立即出现在服务列表，需要等 MCDN agent 重新上报心跳、填充 dial_ips 并更新 updated_at 后，才会在下一个 30 秒窗口内重新出现。

2，人工手动上线

人工发起HTTP请求，基本可以认同为第一种方式，目前基本不使用

```
1  PUT /api/v1/portal/pcdn?name=<hostname>
2  Body: {"is_disabled": false}
```

3，北斗节点心跳恢复

(1) 适用场景：北斗节点超过3分钟未上报心跳包，后续心跳恢复，重新上报心跳。这种情况会通过portal原地进行恢复。

(2) 恢复流程：

portal本身会有定时任务，30s执行一次。刷新mcdn节点服务列表，每次都会从数据库中加载最近3分钟内有更新的节点。

```
1  // dao/mcdn.go:11
2  sql := `updated_at > ? and is_deleted = ? and dial_ips != ? and type = 'v'`
3  m, _ := time.ParseDuration(`-3m`)
4  d.db.Where(sql, time.Now().Add(m), 0, "").Find(&instances.PInstances)
```

MCDN agent 恢复运行 →重新上报拨测结果 →portal 更新 p_instances.updated_at 和 dial_ips →下一个 30 秒缓存刷新周期内节点自动重新出现，全程无任何 API 调用或状态变更。

两个维度的叠加关系：

is_disabled	updated_at / dial_ips	节点是否在服务列表（前端页面展示）
false	3 分钟内有更新 且 dial_ips 非空	在
false	超时 或 dial_ips 为空	不在（心跳失效）
true	任意	不在（is_disabled 过滤发生在 getMCDNs 业务层）

4，blink调用北斗接口进行恢复

（1）适用场景：北斗节点超过3分钟未上报心跳包且未自动恢复、北斗节点一天内被踢下线超过3次，需要blink调用portal接口选择原地恢复节点或者重新交付节点

（2）恢复流程：

重新交付：会把该节点作为全新机器进行交付，走portal POST /pcdns/create接口

原地恢复：blink原地恢复会通过执行脚本，然后等待mcdn恢复心跳。脚本命令是：

```
1 supervisorctl restart portal_pin && python /data/cdn_tools/mcdn_init.py
```

该命令其实是重启portal服务，然后运行mcdn_init脚本，对portal服务进行检查，输出各项状态监控指标（服务类型、hostname、厂商、sid、dns、内核版本、系统语言、时区、v/live类型检查磁盘挂载+BvcApp容器是否运行；mp2sp/mnet类型检查cyberpunk容器是否运行）

BBAN 依赖北斗接口

第一个地址用于周期性同步北斗mcdn -v资源状态，这里只同步在线节点

第三个地址用于周期性同步北斗所有mcdn -v资源，包含在线+离线节点，用于机房划分

```
# 北斗同步mcdn相关配置
portalMCDNResourceAPI = "http://portal.bilibili.co/api/v1/portal/mcdn/resource"
portalMCDNVodNodeAPI = "http://portal.bilibili.co/api/v1/portal/mcdn/nodes"
portalAllMCDNAPI = "https://portal.bilibili.co/api/v1/portal/mcdn/list"
```

```
// GetAllMcdnInfos 读取北斗 mcdn/list 接口获取所有 mcdn 节点 (包括在线、不在线的节点)
Tabnine | Edit | Test | Explain | Document |
func (p *PortalClient) GetAllMcdnInfos() ([]*McdnInfo, error) {
    raw, err := p.RequestPortal(p.PortalAllMCDNAPI)
    if err != nil {
        return nil, err
    }

    rawStr := string(raw)
    code := gjson.Get(rawStr, "code").Int()
    if code != 0 {
        return nil, fmt.Errorf("[GetAllMcdnInfos] response body get non-zero code: %v", code)
    }

    data := gjson.Get(rawStr, "data").Raw
    mcdnAll := gjson.Get(data, "mcdn_all").Raw

    out := make([]*McdnInfo, 0)
    err = json.Unmarshal([]byte(mcdnAll), &out)
    if err != nil {
        return nil, err
    }

    return out, nil
}

// GetAllMCDNInfoFromPortal 读取北斗 portal/mcdn/resource 接口获取所有【在线】状态的节点
Tabnine | Edit | Test | Explain | Document |
func (p *PortalClient) GetAllMCDNInfoFromPortal() (map[int64]*McdnInfo, error) {
    rawData, err := RequestPortalWithAuth(p.PortalMCDNResourceAPI, p.PortalSecret, p.PortalSignature)
    if err != nil {
        return nil, fmt.Errorf("request %s error: %w", p.PortalMCDNResourceAPI, err)
    }

    mcdnsRaw := gjson.Get(rawData, "mcdn_all").Raw
    mcdns := make([]*McdnInfo, 0)
    err = json.Unmarshal([]byte(mcdnsRaw), &mcdns)
    if err != nil {
        return nil, err
    }

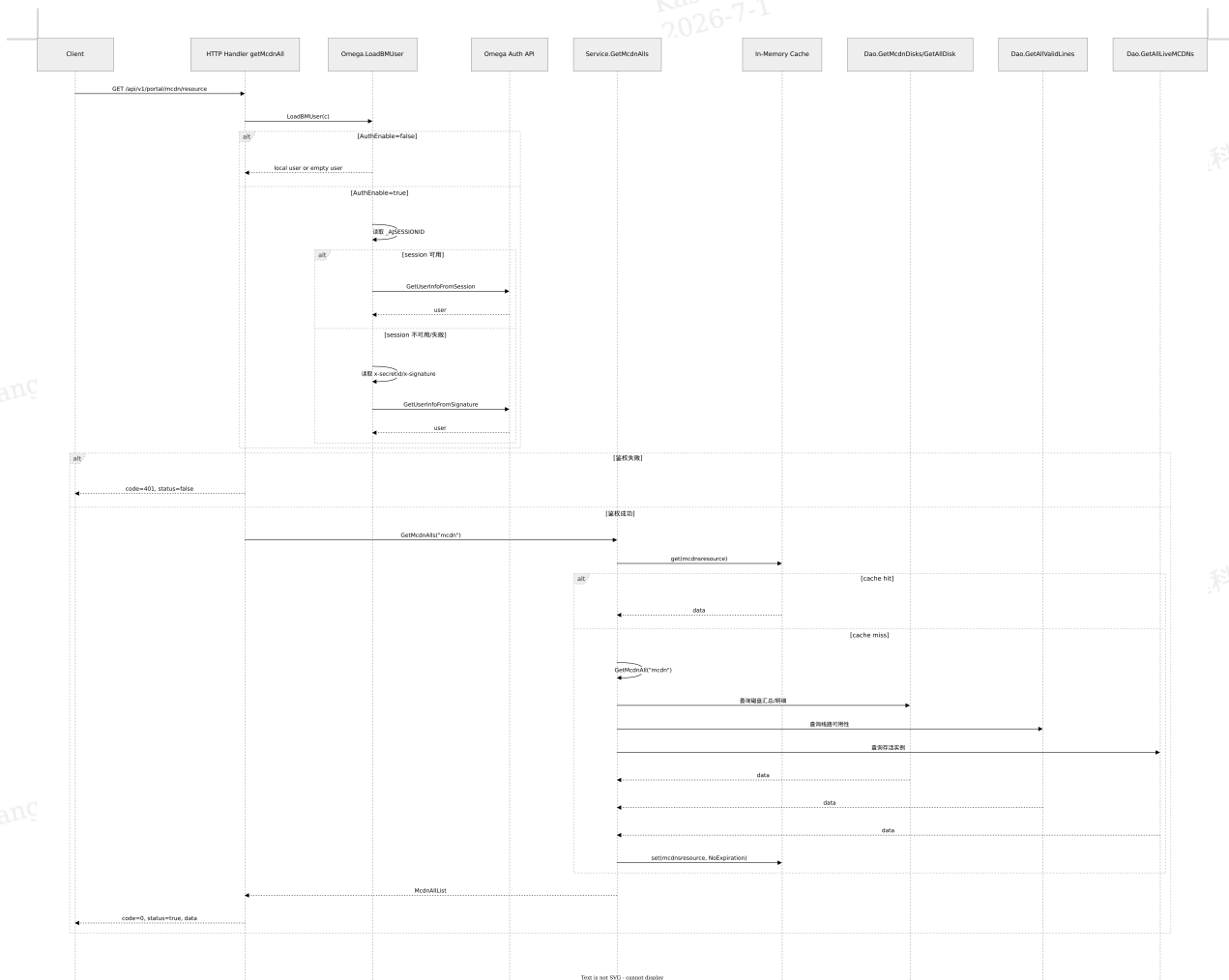
    return McdnInfos2Map(mcdns), nil
}
```

对应北斗中的这两个接口

```
// mcdn域名降级
k := v1.Group("mcdn")
k.GET("/off_domain/list", getMcdnDomains) // 所有
k.GET("/off_domain/tip/:key", getMcdnDomainSuggestion) // 获取提示
k.POST("/domain/off", addMcdnDomains) // 新增
k.PUT("/domain/off/:id", updateMcdnDomains) // 批量开启/关闭
k.DELETE("/off_domain", deleteMcdnDomain) // 删除
k.GET("/off_domain", getMcdnOffDomain) // 所有sid
k.GET("/live/off_domain/list", getVideoMcdnDomains) // 直播
k.GET("/video/off_domain/list", getVideoMcdnDomains) // 点播
k.GET("/resource", getMcdnAll) // 全部在线节点 (含基础信息 BVC-HUB 依赖)
k.GET("/list", listMcdn) // 全部节点
```

具体接口逻辑补充 @similar(liurutong)

一，获取全部在线mcdn节点，/mcdn/resource



1，鉴权

2，调用GetMcdnAlls方法，传入参数mcdn

2.1，读取go-cache本地缓存，获取全部的mcdn节点

2.2，缓存miss，调用GetMcdnAll方法，获取成功后回写缓存。同时服务启动时会开启定时任务，每30s刷新一次缓存

2.3，读数据库，查询mcdn数据

2.3.1，调用dao层方法，读mysql，**mcdn_disks**表，获取过去10分钟内，mcdn节点磁盘数据（包括节点挂载磁盘数量、cpu利用率

内存利用率、服务名、磁盘状态、出入带宽、1分钟/5分钟/15分钟负载等）；

2.3.2，读取clickhouse，**ads_tcp_conn_status_rt**表，获取过去5分钟内所有节点线路监控信息，按照ip聚合数量

2.3.3，读取mysql，**p_instances**表，获取过去6分钟内所有在线的mcdn节点列表

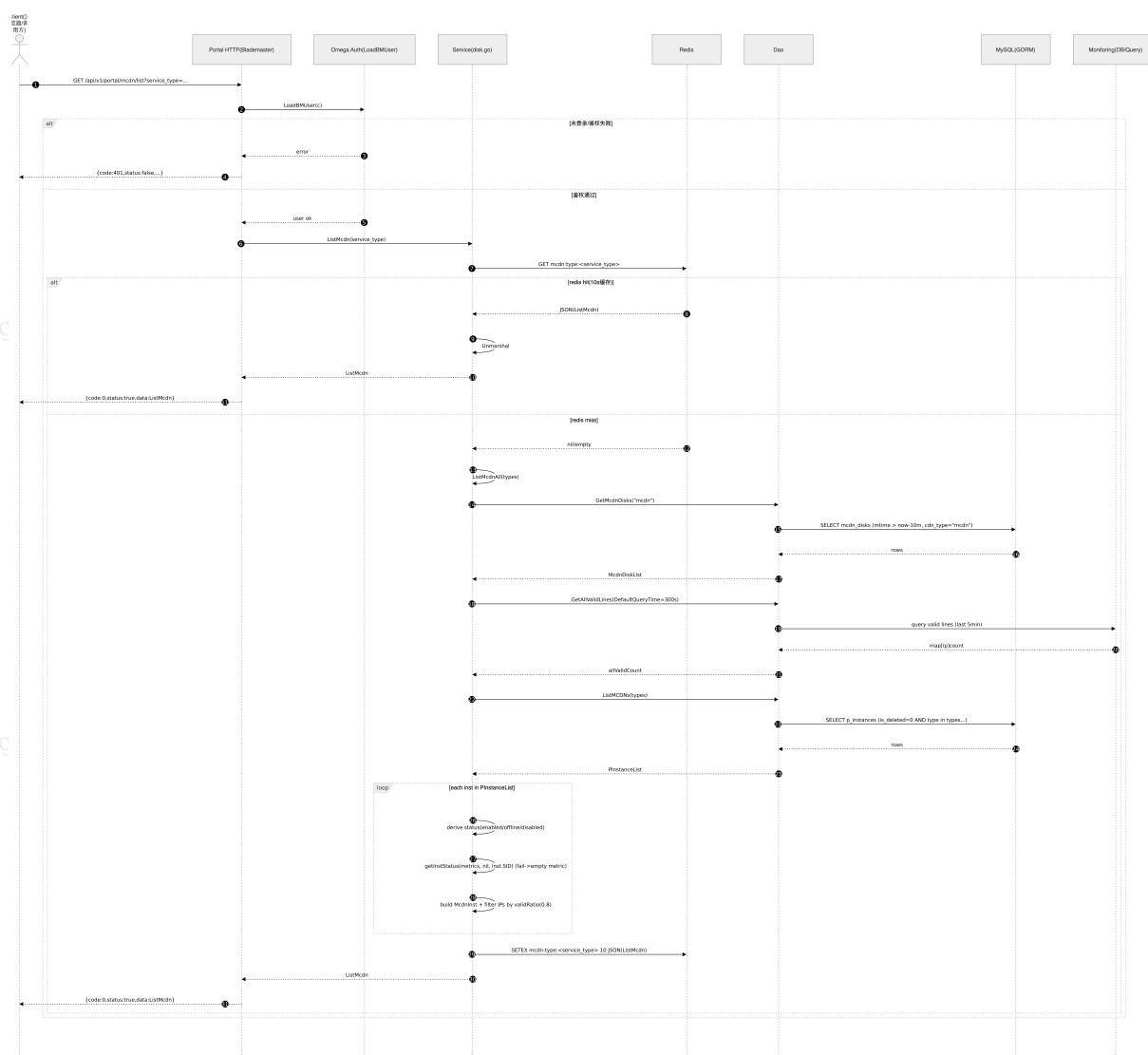
2.3.4，读取mysql，**mcdn_info_disks**表，传入参数mcdn，获取过去10分钟内所有的磁盘信息（包括盘符、挂在目录、文件系统、总容量、剩余容量、使用量、使用率、读写字节数、读写次数、读写时间、磁盘状态等）；

2.3.5，获取在线mcdn节点列表中没有被禁用的线路总数量，计算validRatio，节点线路监控数量/mcdn节点列表中没有禁用的线路数量

2.3.6，获取当前时间作为版本号，遍历所有在线的mcdn实例：

e, 获取所有探测得到的线路数量, 如果监控数据中能够使用的比例不足0.8, 那么不采信这次数据; 否则从监控中判断探测的ip和ipv6是否可用, 可用则添加到mcdn的ip和ipv6中

二，获取全部mcdn节点，/mcdn/list



3.3, 获取在线mcdn节点列表中没有被禁用的线路总数量, 计算validRatio = 节点线路监

控数量/mcdn节点列表中没有禁用的线路数量

3.4，遍历所有的mcdn实例

3.4.1，更新节点的状态，主机超过6分钟没更新，被认为是下线

3.4.2，获取节点的磁盘信息，schedIsp、unschedulableProvince

3.4.3，组装mcdn结构数据

3.4.4，获取所有探测得到的线路数量，如果监控数据中能够使用的比例不足0.8，那么不采信这次数据；否则从监控中判断探测的ip和ipv6是否可用，可用则添加到mcdn的ip和ipv6中

3.5，返回所有的mcdn数据

4，mcdn数据回写缓存，过期时间10秒

三、数据写入

1，mcdn_disks：主要通过/portal/v1/vcdn/disk/:name、/portal/v1/:name两个接口写入

```
259 {
260     // mcdn域名降级
261     k := v1.Group("mcdn")
262     k.GET("/off_domain/list", getMcdnDomains) // 所有
263     k.GET("/off_domain/tip/:key", getMcdnDomainSuggestion) // 获取提示
264     k.POST("/domain/off", addMcdnDomains) // 新增
265     k.PUT("/domain/off/:id", updateMcdnDomains) // 批量开启/关闭
266     k.DELETE("/off_domain", deleteMcdnDomain) // 删除
267     k.GET("/off_domain", getMcdnOffDomain) // 所有sid
268     k.GET("/live/off_domain/list", getVideoMcdnDomains) // 直播
269     k.GET("/video/off_domain/list", getVideoMcdnDomains) // 点播
270     k.GET("/resource", getMcdnAll) // 全部在线节点 (含基础信息 BVC-HUB 依赖)
271     k.GET("/list", listMcdn) // 全部节点
272 }
273 {
274     k := v1.Group("mcdn/disk")
275     k.GET("/list", getDiskList)
276     k.PUT("/:name", updateMcdnDisks) // 张航航, 4 years ago • disk
277     //k.POST("/task/:id", updateMcdnDisks)
278 }
279
280 //vcdn磁盘信息
281 vcdn.GET("/disk/id", getDiskInfo)
282 vcdn.PUT("/disk/:name", updateMcdnDisks) // yulia, 4 years ago • 新增vcdn磁盘上报接口
283 vcdn.GET("/resource", getVcdnDiskAll)
284 vcdn.GET("/disk/list", getDiskList) // 磁盘信息接口
```

2，mcdn_info_disks：写入入口和上面相同

3，p_instances：写入入口是PCDN/MCDN设备管理接口，/portal/v1/pdcn，/portal/v1/id

```
232 {
233   // 新版pcdn 管理
234   i := v1.Group("pcdn")
235   i.POST("", addPInstance) // 添加设备 高夫宝, 2 years ago • feat: 新增设备上下线接口和心跳异常恢复通知接口
236   i.POST("/:id/account", addPCDNAccount) //
237   i.GET("", getPInstances) // 查看设备, 包括离线
238   i.GET("/:id", getPInstance) //
239   i.DELETE("/:id", deletePInstance) //
240   i.PUT("/:id", updatePInstance) // 高夫宝, 2 years ago • feat: 新增设备上下线接口和心跳异常恢复通知接口
241   i.PUT("/:id/rms", syncToRMS) //
242   i.PUT("", updatePCDNs) // update by query
243   i.GET("/:id/label", getPcdnLabels) //
244   i.DELETE("/:id/label/:lid", removePcdnLabels) //
245 }
```

4, ads_tcp_conn_status_rt : 通过采集监控上报到databus, 再由外部链路落库到CH